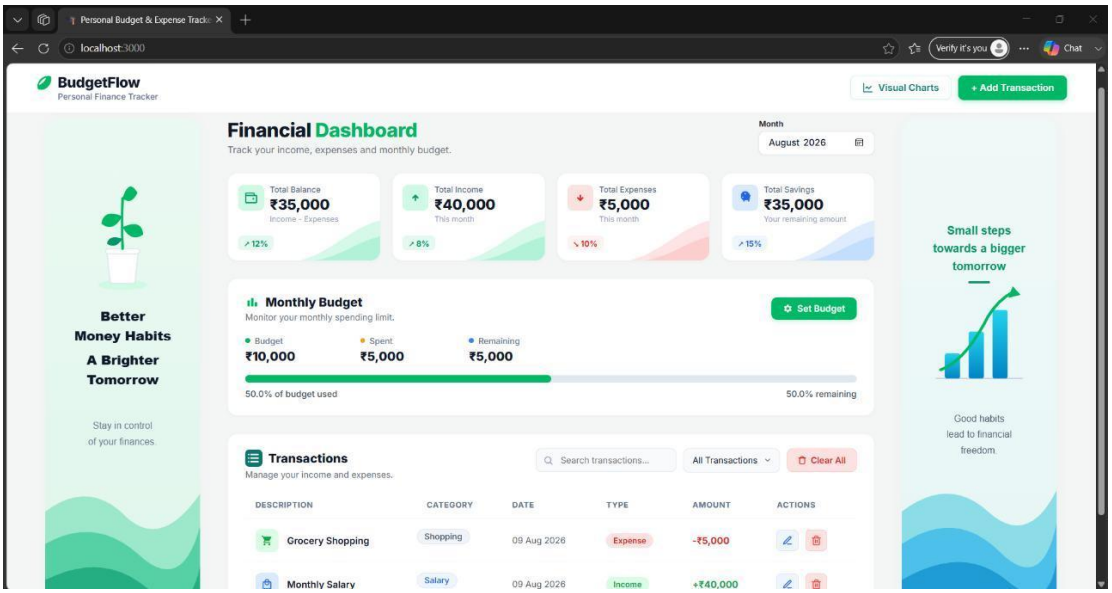
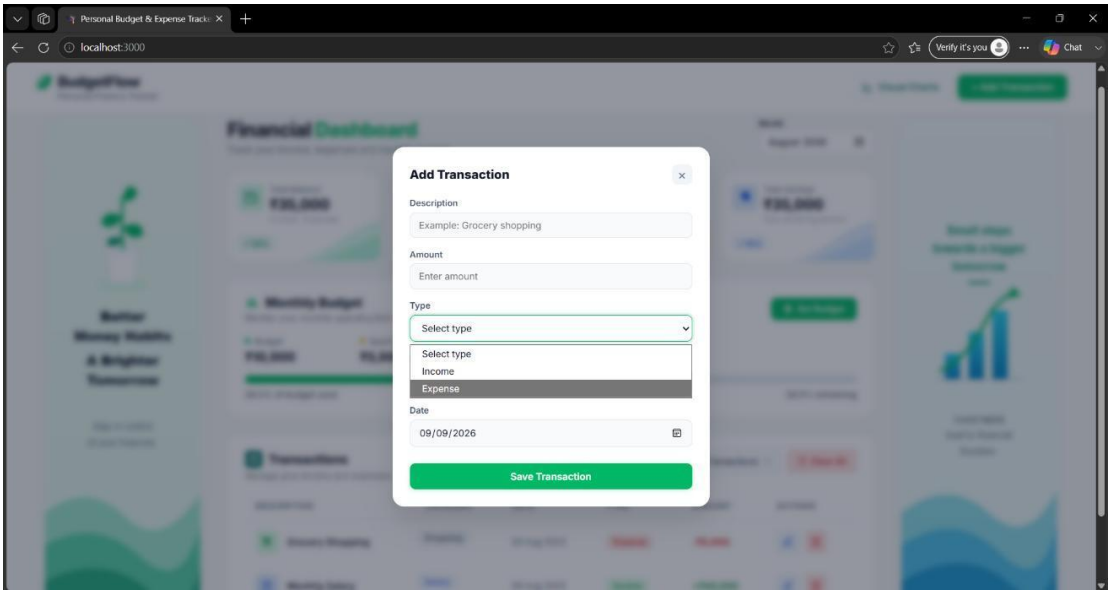
U21AD502 – APPLICATION DEVELOPMENT – ASSIGNMENT 1

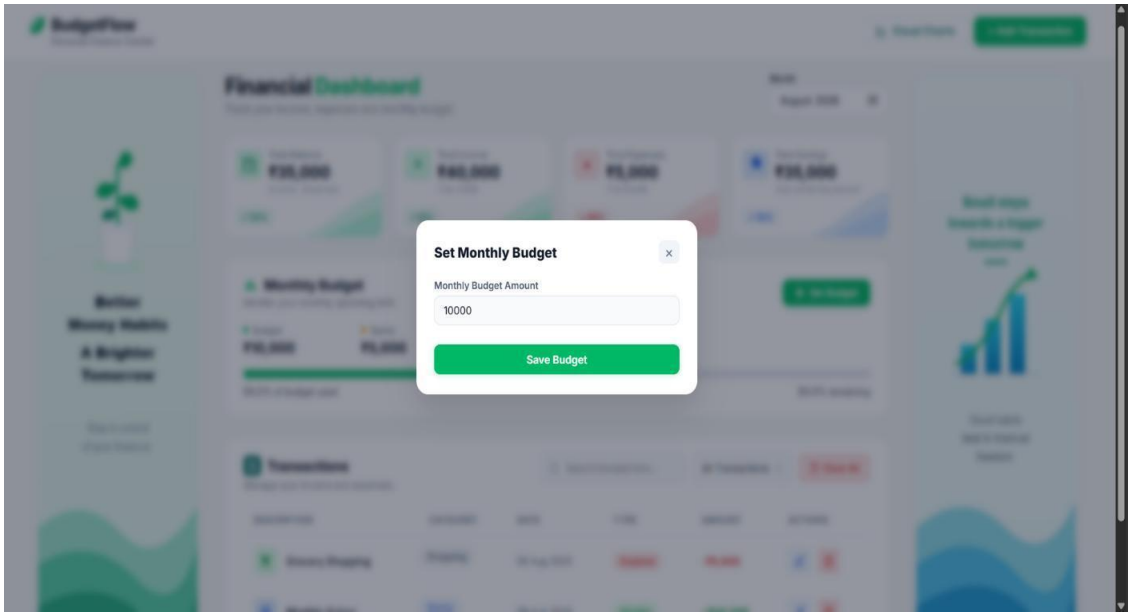
NAME: JAIWANT KARRUN SA ROLLNO: 24AD048 SUBMISSION DATE: 10.09.2026

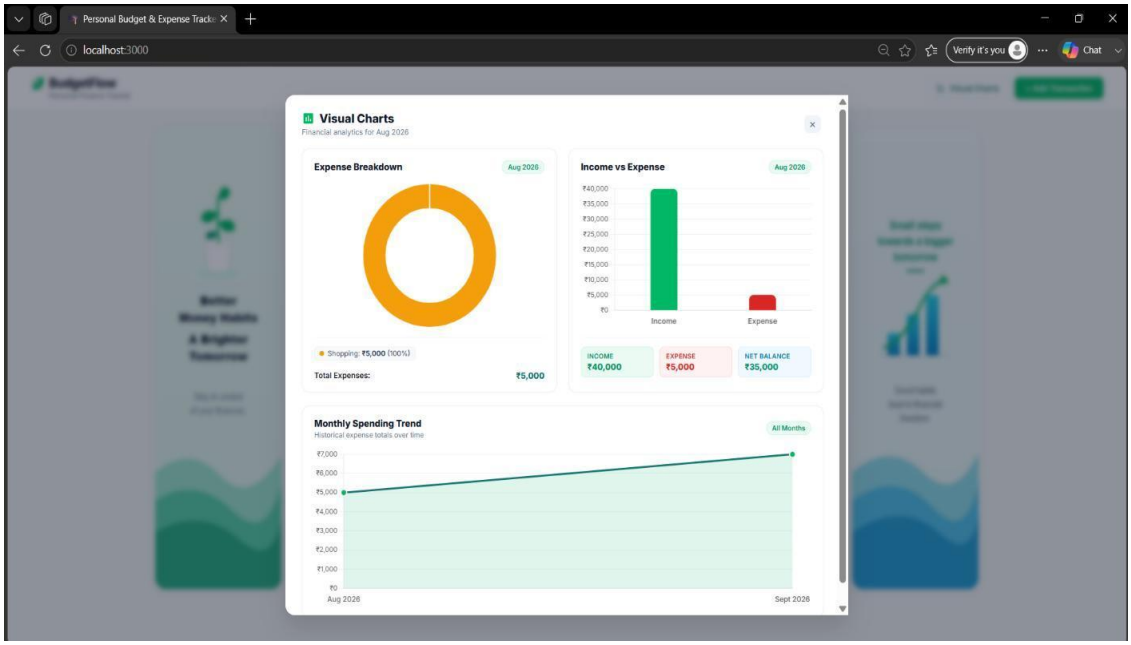
MARKS			
1. User Interface Design & Layout - HTML Structure (15)	2. CSS Implementation & JavaScript Functionality and Validation (15)	3. Server-side programming (10)	TOTAL (40)

UI Design:






Financial Dashboard
Set Monthly Budget
Monthly Budget Amount
10000
Save Budget


Personal Budget & Expense Tracker
localhost:3000
Verify it's you
Chat
Visual Charts
Financial analytics for Aug 2026
Expense Breakdown
Aug 2026
Shopping: ₹5,000 (100%)
Total Expenses:
₹5,000
Income vs Expense
Aug 2026
Income
Expense
INCOME
₹40,000
EXPENSE
₹5,000
NET BALANCE
₹35,000
Monthly Spending Trend
Historical expense totals over time
All Months
Aug 2026
Sept 2026

App.jsx:

```
import React, { useState, useEffect, useMemo } from "react";
import Navbar from "./components/Navbar";
import SummaryCards from "./components/SummaryCards";
import BudgetSection from "./components/BudgetSection";
import TransactionsSection from "./components/TransactionsSection";
import TransactionModal from "./components/TransactionModal";
import BudgetModal from "./components/BudgetModal";
import VisualChartsModal from "./components/VisualChartsModal";
import {
  getTransactions,
  addTransaction,
  updateTransaction,
  deleteTransaction,
  clearAllTransactions,
  getBudget,
  saveBudget
} from "./services/api";

export default function App() {
  const [transactions, setTransactions] = useState([]);
  const [monthlyBudget, setMonthlyBudget] = useState(0);

  // Default to current year-month
  const [selectedMonth, setSelectedMonth] = useState(() => {
    const now = new Date();
    return `${now.getFullYear()}-${String(now.getMonth() + 1).padStart(2, "0")}`;
  });

  const [searchTerm, setSearchTerm] = useState("");
  const [typeFilter, setTypeFilter] = useState("all");

  // Modal state
  const [isTransactionModalOpen, setIsTransactionModalOpen] = useState(false);
  const [isBudgetModalOpen, setIsBudgetModalOpen] = useState(false);
  const [isChartsModalOpen, setIsChartsModalOpen] = useState(false);
  const [editingTransaction, setEditingTransaction] = useState(null);

  // Initial load from MongoDB backend
  useEffect(() => {
    async function loadInitialData() {
      try {
        const [txRes, budgetRes] = await Promise.all([
          getTransactions(),
          getBudget()
        ]);

        if (txRes && txRes.transactions) {
          setTransactions(txRes.transactions);
        }

        if (budgetRes && typeof budgetRes.monthlyBudget === "number") {
          setMonthlyBudget(budgetRes.monthlyBudget);
        }
      } catch (error) {
        console.error("Failed to load initial data:", error);
        alert(
          "Unable to connect to the server.\n\nPlease make sure the Node.js/MongoDB server is running."
        );
      }
    }

    loadInitialData();
  }, []);

  // Global Escape key listener to close active modals
```

```
useEffect(() => {
  const handleKeyDown = e => {
    if (e.key === "Escape") {
      setIsTransactionModalOpen(false);
      setIsBudgetModalOpen(false);
      setIsChartsModalOpen(false);
    }
  };

  window.addEventListener("keydown", handleKeyDown);
  return () => window.removeEventListener("keydown", handleKeyDown);
}, []);

// Monthly summary calculation (Income, Expense, Balance, Savings)
const { totalIncome, totalExpense, balance } = useMemo(() => {
  const monthlyTxs = transactions.filter(t => {
    return !selectedMonth || (t.date && String(t.date).startsWith(selectedMonth));
  });

  let income = 0;
  let expense = 0;

  monthlyTxs.forEach(t => {
    const amt = Number(t.amount) || 0;
    if (t.type === "income") income += amt;
    else if (t.type === "expense") expense += amt;
  });

  return {
    totalIncome: income,
    totalExpense: expense,
    balance: income - expense
  };
}, [transactions, selectedMonth]);

// Filtered and sorted transactions list
const filteredTransactions = useMemo(() => {
  const s = searchTerm.toLowerCase().trim();

  return transactions
    .filter(t => {
      const desc = String(t.description || "").toLowerCase();
      const cat = String(t.category || "").toLowerCase();

      const matchesSearch = !s || desc.includes(s) || cat.includes(s);
      const matchesType = typeFilter === "all" || t.type === typeFilter;
      const matchesMonth =
        !selectedMonth ||
        (t.date && String(t.date).startsWith(selectedMonth));

      return matchesSearch && matchesType && matchesMonth;
    })
    .sort((a, b) => new Date(b.date) - new Date(a.date));
}, [transactions, searchTerm, typeFilter, selectedMonth]);

// Save transaction handler (Create / Update)
const handleSaveTransaction = async transactionData => {
  try {
    if (editingTransaction) {
      const res = await updateTransaction(
        editingTransaction.id,
        transactionData
      );

      setTransactions(prev =>
        prev.map(t =>
          String(t.id) === String(editingTransaction.id)
```

```
                ? res.transaction
                : t
            )
          );

          alert("Transaction updated successfully.");
        } else {
          const res = await addTransaction(transactionData);
          setTransactions(prev => [res.transaction, ...prev]);

          alert("Transaction added successfully.");
        }

        setIsTransactionModalOpen(false);
        setEditingTransaction(null);
      } catch (error) {
        console.error("Error saving transaction:", error);
        alert(error.message || "Unable to save transaction.");
      }
    };

    // Delete single transaction
    const handleDeleteTransaction = async id => {
      if (!id) return;

      const confirmDelete = window.confirm(
        "Are you sure you want to delete this transaction?"
      );
      if (!confirmDelete) return;

      try {
        await deleteTransaction(id);
        setTransactions(prev =>
          prev.filter(t => String(t.id) !== String(id))
        );
      } catch (error) {
        console.error("Error deleting transaction:", error);
        alert(error.message || "Unable to delete transaction.");
      }
    };

    // Clear all transactions
    const handleClearAll = async () => {
      if (!transactions || transactions.length === 0) {
        alert("There are no transactions to clear.");
        return;
      }

      const confirmClear = window.confirm(
        "Are you sure you want to clear all transactions? This action cannot be undone."
      );
      if (!confirmClear) return;

      try {
        await clearAllTransactions();
        setTransactions([]);
      } catch (error) {
        console.error("Error clearing transactions:", error);
        alert(error.message || "Unable to clear transactions.");
      }
    };

    // Save budget
    const handleSaveBudget = async amount => {
      try {
        const res = await saveBudget(amount);
        setMonthlyBudget(Number(res.monthlyBudget));
        setIsBudgetModalOpen(false);
```

```
      alert("Monthly budget saved successfully.");
    } catch (error) {
      console.error("Error saving budget:", error);
      alert(error.message || "Unable to save budget.");
    }
  };

  return (
    <div>
      {/* NAVBAR */}
      <Navbar
        onOpenCharts={() => setIsChartsModalOpen(true)}
        onOpenAddTransaction={() => {
          setEditingTransaction(null);
          setIsTransactionModalOpen(true);
        }}
      />

      {/* MAIN DASHBOARD */}
      <main className="container">
        {/* WELCOME / HEADER */}
        <section className="welcome">
          <div>
            <h1>Financial Dashboard</h1>
            <p>Track your income, expenses and monthly budget.</p>
          </div>

          <div className="month-box">
            <label htmlFor="monthFilter">Month</label>
            <input
              type="month"
              id="monthFilter"
              title="Select a month"
              value={selectedMonth}
              onChange={e => setSelectedMonth(e.target.value)}
            />
          </div>
        </section>

        {/* SUMMARY CARDS */}
        <SummaryCards
          totalIncome={totalIncome}
          totalExpense={totalExpense}
          balance={balance}
        />

        {/* BUDGET SECTION */}
        <BudgetSection
          monthlyBudget={monthlyBudget}
          totalExpense={totalExpense}
          onOpenBudgetModal={() => setIsBudgetModalOpen(true)}
        />

        {/* TRANSACTIONS SECTION */}
        <TransactionsSection
          transactions={filteredTransactions}
          searchTerm={searchTerm}
          onSearchChange={setSearchTerm}
          typeFilter={typeFilter}
          onTypeFilterChange={setTypeFilter}
          onClearAll={handleClearAll}
          onEditTransaction={tx => {
            setEditingTransaction(tx);
            setIsTransactionModalOpen(true);
          }}
          onDeleteTransaction={handleDeleteTransaction}
        />
      </main>
```

```
        {/* MODALS */}
        <TransactionModal
          isOpen={isTransactionModalOpen}
          onClose={() => {
            setIsTransactionModalOpen(false);
            setEditingTransaction(null);
          }}
          onSave={handleSaveTransaction}
          editingTransaction={editingTransaction}
        />

        <BudgetModal
          isOpen={isBudgetModalOpen}
          onClose={() => setIsBudgetModalOpen(false)}
          currentBudget={monthlyBudget}
          onSaveBudget={handleSaveBudget}
        />

        <VisualChartsModal
          isOpen={isChartsModalOpen}
          onClose={() => setIsChartsModalOpen(false)}
          transactions={transactions}
          selectedMonth={selectedMonth}
        />
      </div>
    );
}
```

main.jsx:

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

server.js:

```
const path = require("path");
require("dotenv").config({ path: path.join(__dirname, ".env") });
require("dotenv").config(); // Fallback to root .env if present

const express = require("express");
const cors = require("cors");
const mongoose = require("mongoose");
const fs = require("fs");

const Transaction = require("./models/Transaction");
const Budget = require("./models/Budget");

const app = express();
const PORT = process.env.PORT || 5000;
const MONGODB_URI =
  process.env.MONGODB_URI || "mongodb://127.0.0.1:27017/budgetflow";

// ==========================================
// MIDDLEWARE
// ==========================================

app.use(cors());
app.use(express.json());

// Serve static React build if it exists
const clientDistPath = path.join(__dirname, "..", "client", "dist");

if (fs.existsSync(clientDistPath)) {
  app.use(express.static(clientDistPath));
}

// ==========================================
// MONGODB CONNECTION & AUTO-MIGRATION
// ==========================================

async function connectDB() {
  try {
    await mongoose.connect(MONGODB_URI);
    console.log("Connected to MongoDB successfully at:", MONGODB_URI);

    // Auto-migration from existing budget-data.json if MongoDB is empty
    await autoMigrateFromJson();
  } catch (error) {
    console.error("MongoDB connection error:", error.message);
    console.log("Please ensure MongoDB is running on your machine.");
  }
}

async function autoMigrateFromJson() {
  try {
    const count = await Transaction.countDocuments();
    if (count === 0) {
      let dataFile = path.join(__dirname, "data", "budget-data.json");
      if (!fs.existsSync(dataFile)) {
        dataFile = path.join(__dirname, "..", "data", "budget-data.json");
      }

      if (fs.existsSync(dataFile)) {
        const raw = fs.readFileSync(dataFile, "utf8");
        const jsonData = JSON.parse(raw);

        if (Array.isArray(jsonData.transactions) && jsonData.transactions.length > 0) {
          await Transaction.insertMany(jsonData.transactions);
          console.log(
```

```
            `[Migration] Successfully imported ${jsonData.transactions.length} transactions into MongoDB.`
          );
        }

        if (typeof jsonData.monthlyBudget === "number" && jsonData.monthlyBudget > 0) {
          await Budget.findOneAndUpdate(
            { key: "current_budget" },
            { monthlyBudget: jsonData.monthlyBudget },
            { upsert: true, returnDocument: "after" }
          );
          console.log(
            `[Migration] Imported monthly budget (₹${jsonData.monthlyBudget}) into MongoDB.`
          );
        }
      }
    }
  } catch (err) {
    console.warn("[Migration Notice] Could not auto-migrate from JSON:", err.message);
  }
}

connectDB();

// ==========================================
// TRANSACTION VALIDATION HELPER
// ==========================================

function validateTransaction(data) {
  if (typeof data.description !== "string" || data.description.trim() === "") {
    return "Description is required.";
  }

  if (
    typeof data.amount !== "number" ||
    !Number.isFinite(data.amount) ||
    data.amount <= 0
  ) {
    return "Amount must be greater than zero.";
  }

  if (data.type !== "income" && data.type !== "expense") {
    return "Transaction type must be income or expense.";
  }

  if (typeof data.category !== "string" || data.category.trim() === "") {
    return "Category is required.";
  }

  if (
    typeof data.date !== "string" ||
    !/^\d{4}-\d{2}-\d{2}$/.test(data.date)
  ) {
    return "Valid date is required.";
  }

  return null;
}

// ==========================================
// GET TRANSACTIONS
// ==========================================

app.get("/api/transactions", async function (req, res) {
  try {
    const transactions = await Transaction.find().sort({ date: -1, createdAt: -1 });

    res.status(200).json({
```

```
      success: true,
      transactions: transactions.map(t => ({
        id: t.id,
        description: t.description,
        amount: t.amount,
        type: t.type,
        category: t.category,
        date: t.date
      }))
    });
  } catch (error) {
    console.error("Error retrieving transactions:", error);
    res.status(500).json({
      success: false,
      message: "Unable to retrieve transactions from database."
    });
  }
});

// ==========================================
// ADD TRANSACTION
// ==========================================

app.post("/api/transactions", async function (req, res) {
  try {
    const amount = Number(req.body.amount);

    const transactionData = {
      description: req.body.description,
      amount,
      type: req.body.type,
      category: req.body.category,
      date: req.body.date
    };

    const validationError = validateTransaction(transactionData);
    if (validationError) {
      return res.status(400).json({
        success: false,
        message: validationError
      });
    }

    const newId = Date.now();
    const createdTransaction = await Transaction.create({
      id: newId,
      description: transactionData.description.trim(),
      amount: transactionData.amount,
      type: transactionData.type,
      category: transactionData.category.trim(),
      date: transactionData.date
    });

    res.status(201).json({
      success: true,
      message: "Transaction added successfully.",
      transaction: {
        id: createdTransaction.id,
        description: createdTransaction.description,
        amount: createdTransaction.amount,
        type: createdTransaction.type,
        category: createdTransaction.category,
        date: createdTransaction.date
      }
    });
  } catch (error) {
    console.error("Error adding transaction:", error);
    res.status(500).json({
      success: false,
```

```
      message: "Server error while adding transaction."
    });
  }
});

// ==========================================
// UPDATE TRANSACTION
// ==========================================

app.put("/api/transactions/:id", async function (req, res) {
  try {
    const targetId = Number(req.params.id);
    const amount = Number(req.body.amount);

    const transactionData = {
      description: req.body.description,
      amount,
      type: req.body.type,
      category: req.body.category,
      date: req.body.date
    };

    const validationError = validateTransaction(transactionData);
    if (validationError) {
      return res.status(400).json({
        success: false,
        message: validationError
      });
    }

    const updatedTransaction = await Transaction.findOneAndUpdate(
      { id: targetId },
      {
        description: transactionData.description.trim(),
        amount: transactionData.amount,
        type: transactionData.type,
        category: transactionData.category.trim(),
        date: transactionData.date
      },
      { returnDocument: "after" }
    );

    if (!updatedTransaction) {
      return res.status(404).json({
        success: false,
        message: "Transaction not found."
      });
    }

    res.status(200).json({
      success: true,
      message: "Transaction updated successfully.",
      transaction: {
        id: updatedTransaction.id,
        description: updatedTransaction.description,
        amount: updatedTransaction.amount,
        type: updatedTransaction.type,
        category: updatedTransaction.category,
        date: updatedTransaction.date
      }
    });
  } catch (error) {
    console.error("Error updating transaction:", error);
    res.status(500).json({
      success: false,
      message: "Server error while updating transaction."
    });
  }
});
```

```
// ========================================
// DELETE ALL TRANSACTIONS (CLEAR ALL)
// ========================================

app.delete("/api/transactions", async function (req, res) {
  try {
    await Transaction.deleteMany({});

    res.status(200).json({
      success: true,
      message: "All transactions cleared successfully."
    });
  } catch (error) {
    console.error("Error clearing transactions:", error);
    res.status(500).json({
      success: false,
      message: "Server error while clearing transactions."
    });
  }
});

// ========================================
// DELETE SINGLE TRANSACTION
// ========================================

app.delete("/api/transactions/:id", async function (req, res) {
  try {
    const targetId = Number(req.params.id);

    const deleted = await Transaction.findOneAndDelete({ id: targetId });

    if (!deleted) {
      return res.status(404).json({
        success: false,
        message: "Transaction not found."
      });
    }

    res.status(200).json({
      success: true,
      message: "Transaction deleted successfully."
    });
  } catch (error) {
    console.error("Error deleting transaction:", error);
    res.status(500).json({
      success: false,
      message: "Server error while deleting transaction."
    });
  }
});

// ========================================
// GET BUDGET
// ========================================

app.get("/api/budget", async function (req, res) {
  try {
    const budgetDoc = await Budget.findOne({ key: "current_budget" });

    res.status(200).json({
      success: true,
      monthlyBudget: budgetDoc ? budgetDoc.monthlyBudget : 0
    });
  } catch (error) {
    console.error("Error retrieving budget:", error);
    res.status(500).json({
```

```
      success: false,
      message: "Unable to retrieve budget."
    });
  }
});

// ==========================================
// SAVE BUDGET
// ==========================================

app.post("/api/budget", async function (req, res) {
  try {
    const amount = Number(req.body.amount);

    if (!Number.isFinite(amount) || amount <= 0) {
      return res.status(400).json({
        success: false,
        message: "Budget amount must be greater than zero."
      });
    }

    const budgetDoc = await Budget.findOneAndUpdate(
      { key: "current_budget" },
      { monthlyBudget: amount },
      { upsert: true, returnDocument: "after" }
    );

    res.status(200).json({
      success: true,
      message: "Budget saved successfully.",
      monthlyBudget: budgetDoc.monthlyBudget
    });
  } catch (error) {
    console.error("Error saving budget:", error);
    res.status(500).json({
      success: false,
      message: "Server error while saving budget."
    });
  }
});

// ==========================================
// UNKNOWN API ROUTE
// ==========================================

app.use("/api", function (req, res) {
  res.status(404).json({
    success: false,
    message: "API endpoint not found."
  });
});

// ==========================================
// CLIENT FALLBACK ROUTE
// ==========================================

app.use(function (req, res) {
  if (fs.existsSync(path.join(clientDistPath, "index.html"))) {
    res.sendFile(path.join(clientDistPath, "index.html"));
  } else {
    res.status(404).send("Application build not found.");
  }
});

// ==========================================
// GLOBAL ERROR HANDLER
// ==========================================
```

```
app.use(function (error, req, res, next) {
   console.error("Unexpected error:", error);
   res.status(500).json({
      success: false,
      message: "Internal server error."
   });
});

// ==========================================
// START SERVER
// ==========================================

app.listen(PORT, function () {
   console.log("");
   console.log("======================================");
   console.log(" BudgetFlow Backend Server Started");
   console.log("======================================");
   console.log(`Server:      http://localhost:${PORT}`);
   console.log(`Transactions: http://localhost:${PORT}/api/transactions`);
   console.log(`Budget:      http://localhost:${PORT}/api/budget`);
   console.log(`Database:    ${MONGODB_URI}`);
   console.log("======================================");
});
```

Transaction.js:

```
const mongoose = require("mongoose");

const TransactionSchema = new mongoose.Schema(
  {
    id: {
      type: Number,
      required: true,
      unique: true,
      default: () => Date.now()
    },
    description: {
      type: String,
      required: [true, "Description is required."],
      trim: true
    },
    amount: {
      type: Number,
      required: [true, "Amount is required."],
      min: [0.01, "Amount must be greater than zero."]
    },
    type: {
      type: String,
      required: [true, "Transaction type is required."],
      enum: {
        values: ["income", "expense"],
        message: "Transaction type must be income or expense."
      }
    },
    category: {
      type: String,
      required: [true, "Category is required."],
      trim: true
    },
    date: {
      type: String,
      required: [true, "Valid date is required."],
      match: [/^\d{4}-\d{2}-\d{2}$/, "Date must be in YYYY-MM-DD format."]
    }
  },
  {
    timestamps: true,
    toJSON: {
      transform: function (doc, ret) {
        delete ret.__v;
        return ret;
      }
    }
  }
);

TransactionSchema.index({ date: -1 });

module.exports = mongoose.model("Transaction", TransactionSchema);
```